

1、认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。

没有多人协同开发的经历

Git 用暂存区把“改了什么”和“这次提交要记录什么”分开，用户可以先挑选、审查并构造一个干净、逻辑独立的提交快照。

`git branch` 仅列出本地分支，而 `git branch -a` 会同时列出本地分支和远程分支。

3、在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。

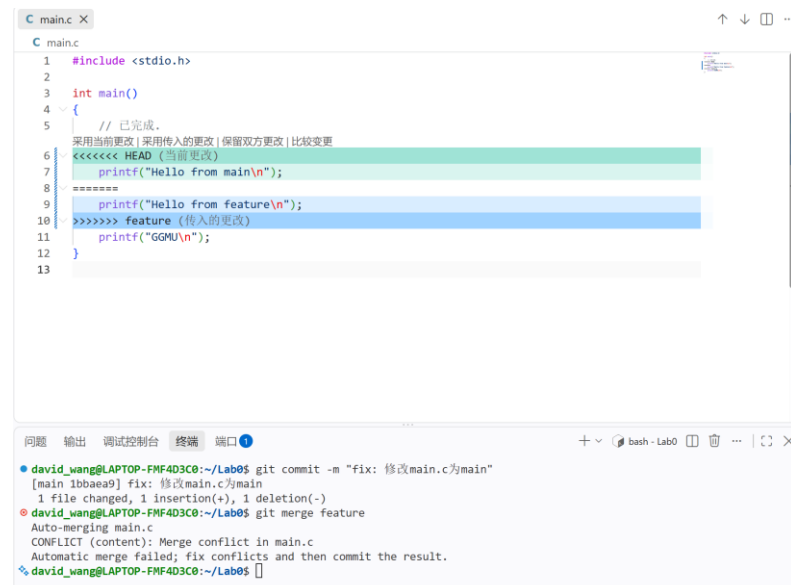
第一篇：这篇文章聚焦于 Git 提交信息的规范化。核心观点是：格式化的 Commit message 能提供更多历史信息，方便快速浏览和查找，还能直接生成 Change log。文章推荐使用 Angular 规范，Commit message 由 Header、Body 和 Footer 三部分组成，其中 Header 是必需的，包含 type、scope 和 subject 三个字段。文章强调，凡是不兼容的变动，必须在 Footer 中以 BREAKING CHANGE 开头进行说明。

第二篇：这篇文章介绍了团队协作中广泛使用的 Gitflow 分支管理模型。其核心目的是保证不同分支各司其职，避免多人协作中的代码冲突和版本混乱。模型包含五种分支：master、develop、feature、release 和 hotfix。文章反复强调一个原则：“从哪里来，最后回到哪里去”，即 feature 从 develop 创建、完成后合并回 develop，hotfix 从 master 创建、修复后合并回 master。

学习 git 是因为不管是自己写代码还是多人协作都需要一个工具来记录代码的修改历史，不然改错了没法回退，多个人改同一个项目也容易互相覆盖导致混乱，而 git 能记录每一次改动并且方便恢复到之前的版本，同时像阮一峰提到的规范 commit message 和 gitflow 分支模型，都是为了在团队协作时让大家清楚谁在哪个分支改了什么以及为什么改，从而避免代码冲突和版本混乱，所以学 git 本质上就是学怎么有序地管理代码的变化，这对个人开发和团队合作来说都是最基础也最实用的技能。

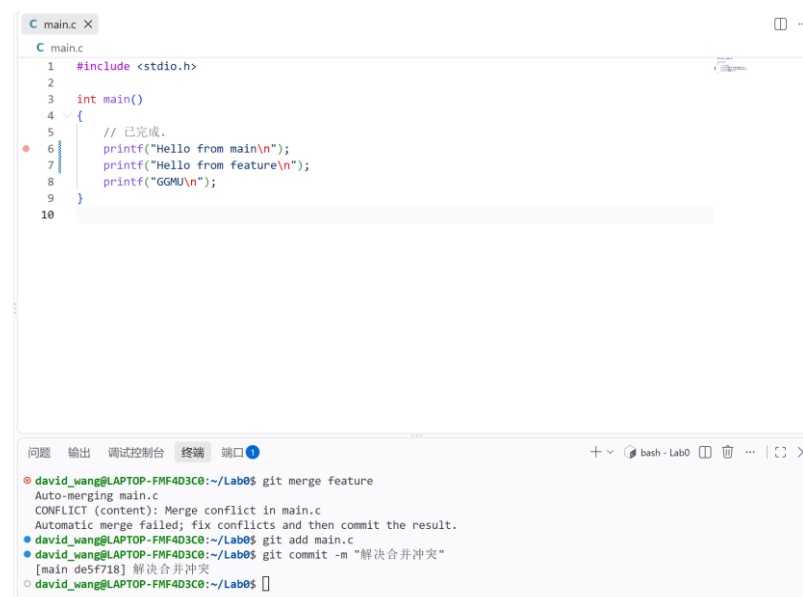
4、学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交

遇到并解决了冲突



```
C main.c X
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // 已完成。
6      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
7      <<<<<< HEAD (当前更改)
8      printf("Hello from main\n");
9      =====
10     printf("Hello from feature\n");
11     >>>>>> feature (传入的更改)
12     printf("GGMU\n");
13 }

问题 输出 调试控制台 终端 端口
david_wang@LAPTOP-FMF4D3C0:~/Lab0$ git commit -m "fix: 修改main.c为main"
[main 1bbaea9] fix: 修改main.c为main
1 file changed, 1 insertion(+), 1 deletion(-)
david_wang@LAPTOP-FMF4D3C0:~/Lab0$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
david_wang@LAPTOP-FMF4D3C0:~/Lab0$
```



```
C main.c X
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // 已完成。
6      printf("Hello from main\n");
7      printf("Hello from feature\n");
8      printf("GGMU\n");
9  }
10

问题 输出 调试控制台 终端 端口
david_wang@LAPTOP-FMF4D3C0:~/Lab0$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
david_wang@LAPTOP-FMF4D3C0:~/Lab0$ git add main.c
david_wang@LAPTOP-FMF4D3C0:~/Lab0$ git commit -m "解决合并冲突"
[main de5f718] 解决合并冲突
david_wang@LAPTOP-FMF4D3C0:~/Lab0$
```

实验步骤：

第一部分：修改 TODO 并首次提交

1. 在 GitHub 使用模板创建个人仓库，在 VS Code 中连接 WSL 并克隆到本地。
2. 在终端配置 Git 全局用户名和邮箱。
3. 打开 main.c，将 TODO 注释修改为 //已完成，并新增一行代码

`printf("GGMU\n");`, 保存文件。

4. 执行 `git add main.c` 暂存修改。
5. 执行 `git commit -m "完成 main.c 中的 TODO 部分"` 提交到本地。
6. 执行 `git push origin main` 推送到 GitHub, 并在网页端确认。

第二部分：分支管理与解决合并冲突

7. 执行 `git checkout -b feature` 新建并切换到 feature 分支。
8. 打开 main.c, 修改代码, 保存后执行 `git add main.c` 和 `git commit -m "feat: 修改 main.c 为 feature"`。
9. 执行 `git checkout main` 切回主分支。
10. 打开 main.c, 在同一位置修改代码, 保存后执行 `git add main.c` 和 `git commit -m "fix: 修改 main.c 为 main"`。
11. 执行 `git merge feature`, 终端报错提示 CONFLICT, 确认发生合并冲突。
12. 打开 main.c, 看到 <<<<<< HEAD、=====、>>>>>> feature 等冲突标记, 截图保存。
13. 在 VS Code 中通过快捷按钮或手动删除冲突标记, 保留最终代码, 保存文件。
14. 执行 `git add main.c` 和 `git commit -m "解决合并冲突"`, 截图记录解决后的状态。
15. 执行 `git push origin main` 推送到远程仓库, 刷新网页端确认提交历史。